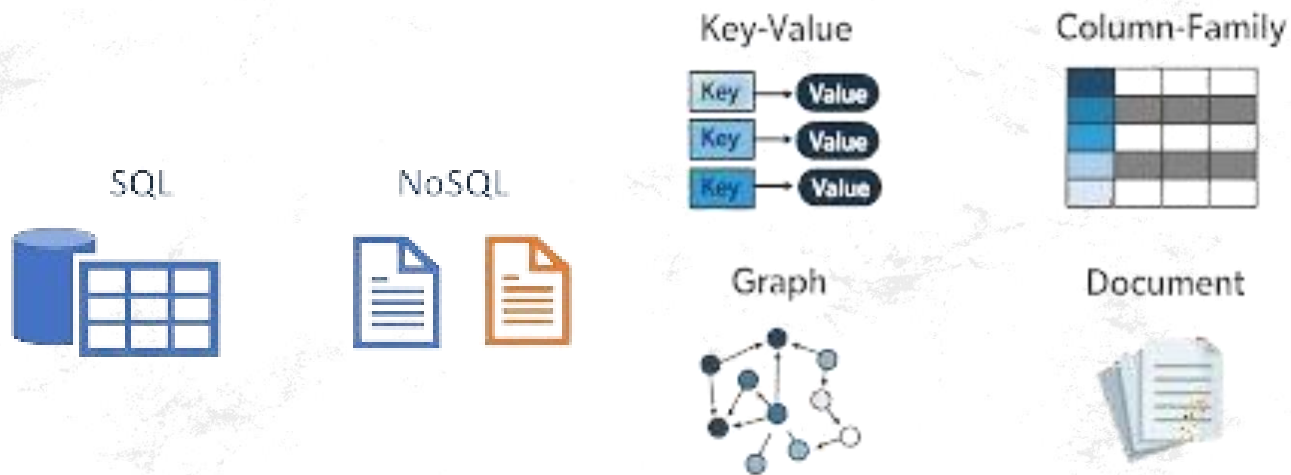


NoSQL



Introduction to NoSQL

NIELIT Chandigarh/Ropar

Agenda

- Introduction to NoSQL
- Types of NoSQL Databases
- MongoDB Overview
- MongoDB CRUD Operations
- MongoDB Aggregation Framework
- Practical Hands-on: CRUD & Aggregation in MongoDB
- Creating a Todo (Notes) app using MongoDB

Introduction to NoSQL

What is NoSQL?

- "NoSQL" stands for "Not Only SQL."
- A non-relational database management system designed to handle large amounts of data and diverse data models.

Why NoSQL?

- Scalability
- Flexibility in data modeling
- High performance for specific use cases
- Designed for distributed systems

Key Characteristics:

- Schema-less or flexible schema
- Horizontal scaling
- Optimized for modern data requirements (e.g., big data, real-time analytics)

Types of NoSQL Databases

1. Document Databases

1. Example: MongoDB
2. Data is stored as JSON-like documents.
3. Best for semi-structured data.

2. Key-Value Stores

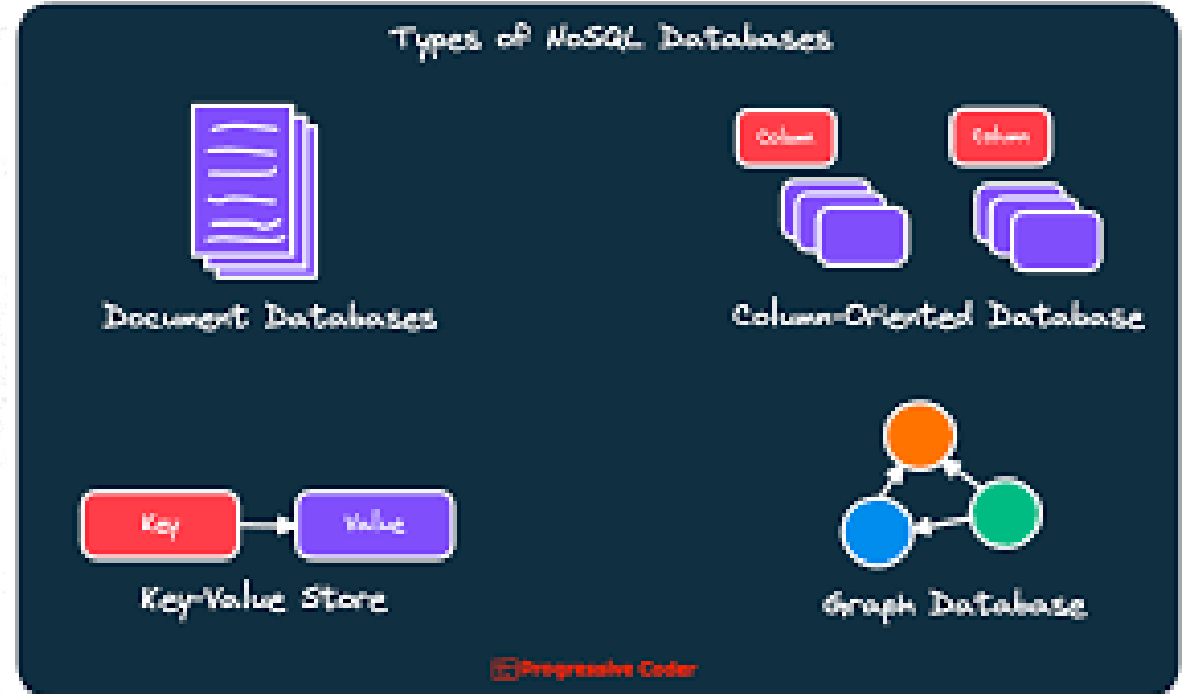
1. Example: Redis, DynamoDB
2. Data is stored as key-value pairs.
3. Ideal for caching and session storage.

3. Column-Oriented Databases

1. Example: Cassandra, HBase
2. Data is stored in columns instead of rows.
3. Suitable for analytical workloads.

4. Graph Databases

1. Example: Neo4j, ArangoDB
2. Data is represented as nodes and edges.
3. Best for relationship-based queries (e.g., social networks).





MongoDB Overview



MongoDB is a popular, open-source, document-oriented NoSQL database.

- Stores data in **JSON-like documents** (BSON).
- Supports **schema-less** data models.
- Provides **horizontal scaling** and high availability through sharding.
- Used by organizations such as Uber, eBay, and Netflix for large-scale applications.

Key Features:

- **Flexible Schema:** Allows for a dynamic and flexible data model.
- **High Availability:** Replica sets for data redundancy and fault tolerance.
- **Sharding:** Distributes data across multiple machines for horizontal scaling.
- **Aggregation Framework:** Powerful data transformation and analysis tool.

MongoDB Overview

Features:

- Schema flexibility
- Horizontal scaling with sharding
- Built-in replication for high availability
- Rich query language

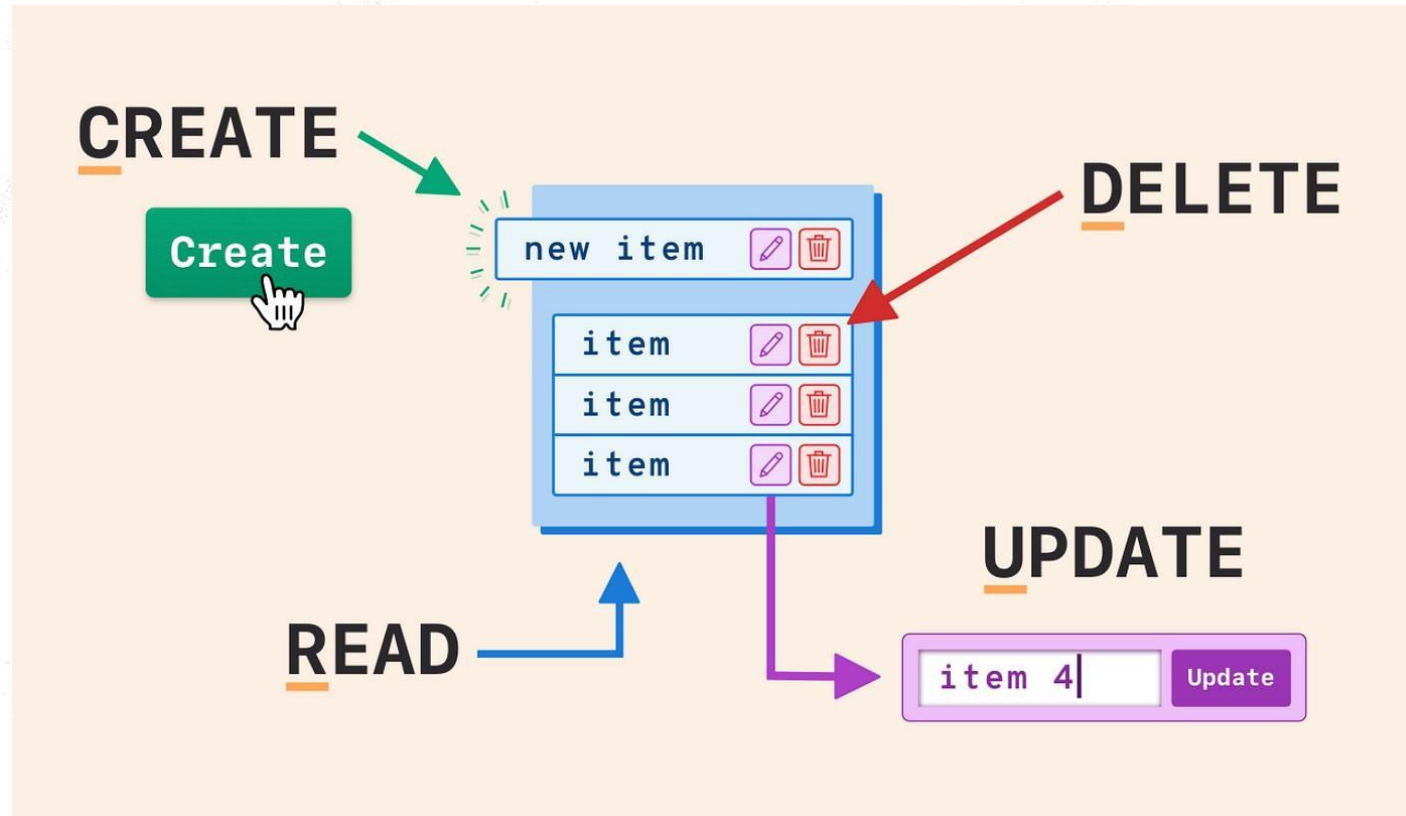
Use Cases:

- Content management
- Real-time analytics
- IoT applications
- Mobile and web applications

Relational DB	MongoDB
database	database
table	collection
row	document
column	field

MongoDB CRUD Operations - Overview

- **CRUD** stands for Create, Read, Update, and Delete, the basic operations in MongoDB.



Create Operation (Insert)

- MongoDB provides methods to insert documents into collections.
 - **insertOne()**: Inserts a single document.
 - **insertMany()**: Inserts multiple documents.

1. Create:

```
db.collection.insertOne({ key: "value" });  
db.collection.insertMany([ { key: "value1" }, { key: "value2" } ]);
```

Example:

// Insert one document

```
db.collection.insertOne({  
  name: "John Doe",  
  age: 30,  
  email: "johndoe@example.com" });
```

// Insert documents

```
db.collection.insertMany([  
  { name: "Alice", age: 25, email:  
    "alice@example.com" },  
  { name: "Bob", age: 35, email:  
    "bob@example.com" } ]);
```


Read Operation (Find)

- MongoDB provides the **find()** method to query the database.

```
db.users.find({ key: "value" });  
db.users.findOne({ key: "value" });
```

Example:

// Find all documents in the collection

```
db.users.find({});
```

// Find specific document(s) based on a condition GT – GREATER THAN

```
db.users.find({ age: { $gt: 30 } });
```

Projection: Specify fields to return:

```
db.users.find({}, { name: 1, email: 1 });
```

Update Operation (Update)

- MongoDB provides methods to modify existing documents.
 - **updateOne()**: Updates a single document.
 - **updateMany()**: Updates multiple documents.

```
db.users.updateOne({ key: "value" }, { $set: { key: "newValue" } });  
db.users.updateMany({ key: "value" }, { $set: { key: "newValue" } });
```

Example:

```
// Update a single document  
db.users.updateOne(  
  { name: "John Doe" },  
  { $set: { age: 31 } } );
```

```
// Update many documents  
db.users.updateMany(  
  { age: { $lt: 30 } },  
  { $set: { status: "young" } } );
```

Delete Operation (Remove)

- MongoDB allows deletion of documents using the `deleteOne()` and `deleteMany()` methods.

```
db.users.deleteOne({ key: "value" });  
db.users.deleteMany({ key: "value" });
```

Example:

// Delete a single document

```
db.users.deleteOne({ name: "John Doe" });
```

// Delete multiple documents

```
db.users.deleteMany({ age: { $lt: 30 } });
```



MongoDB Aggregation Framework



The **Aggregation Framework** is used for data transformation and analysis.

- Supports operations like filtering, grouping, sorting, and reshaping data.

Stages in the aggregation pipeline:

- **\$match**: Filters the data.
- **\$group**: Groups data by a specific field.
- **\$sort**: Sorts data.
- **\$project**: Shapes the output (selects specific fields).
- **\$limit**: Limits the number of documents.

Aggregation Example - Grouping and Summarizing Data

Example: Grouping users by age and calculating the average age.

```
db.users.aggregate([  
  { $group: { _id: "$age", avgAge: { $avg: "$age" } } }  
]);
```

Explanation:

- **\$group:** Groups documents by the field age and calculates the average age.

Aggregation Example - Sorting and Limiting

Example: Sorting users by age and limiting the results to the top 5.

```
db.users.aggregate([  
  { $sort: { age: -1 } },  
  { $limit: 5 }  
]);
```

Explanation:

- \$sort: Sorts the users by the age field in descending order.
- \$limit: Limits the result to the top 5 documents.

MongoDB Aggregation Framework

What is Aggregation?

- A powerful way to perform data processing and analysis.

Stages:

- \$match - Filters documents.
- \$group - Groups documents by a specified key.
- \$project - Shapes the output.
- \$sort - Sorts documents.
- \$limit - Limits the number of documents.

```
db.collection.aggregate([  
  { $match: { status: "active" } },  
  { $group: { _id: "$category", total: { $sum: 1 } } },  
  { $sort: { total: -1 } }  
]);
```

Benefits and Challenges of NoSQL

Benefits:

- High scalability
- Flexible data models
- Fast for specific use cases
- Optimized for big data

Challenges:

- Limited Support for Small Data
- Limited support for complex queries (compared to SQL), Lack of Join-Like Functionality
- Steeper Learning Curve for SQL Users
- Possible consistency trade-offs (e.g., CAP theorem)
- Backup and Restore Complexity

MongoDB Use Cases

- **Content Management Systems:** Storing and managing unstructured content.
- **E-commerce Platforms:** Product catalogs, user data, shopping cart.
- **Real-Time Analytics:** Storing logs and event data for real-time processing.
- **Social Networks:** Storing and managing user profiles and relationships.
- Artificial intelligence, Edge computing, Internet of things, Mobile, Payments, Serverless development, and Personalization.
- MongoDB is **a good choice for teams** that need to develop software applications quickly and scale data.

Here are some companies that use MongoDB

- **Walmart:** Uses MongoDB to manage its product catalog
- **Google:** Uses MongoDB Atlas in combination with Google Cloud to build intelligent applications
- **Microsoft:** Integrates MongoDB Atlas with Microsoft Azure's AI and data analytics tools
- **IBM:** Customers can use MongoDB Community Edition and MongoDB Enterprise Edition as fully managed databases in the IBM Cloud
- **Zomato:** An Indian company that uses MongoDB
- **Tata Digital:** An Indian company that uses MongoDB
- **Canara HSBC Life Insurance:** An Indian company that uses MongoDB
- **Tata AIG:** An Indian company that uses MongoDB
- **Devnagri:** An Indian company that uses MongoDB

Conclusion

- **MongoDB** is a **powerful**, **flexible**, and **scalable** NoSQL database.
- **CRUD operations** provide basic functionality for creating, reading, updating, and deleting data.
- **Aggregation** enables complex data processing and transformation.
- MongoDB is ideal for applications that require **high scalability**, **flexibility**, and **real-time data analysis**.

Next Steps:

- Set up MongoDB locally or **on a cloud service like Atlas**.
- Practice CRUD operations and aggregation queries.



Flask



Create Live Project

NIELIT Chandigarh/Ropar

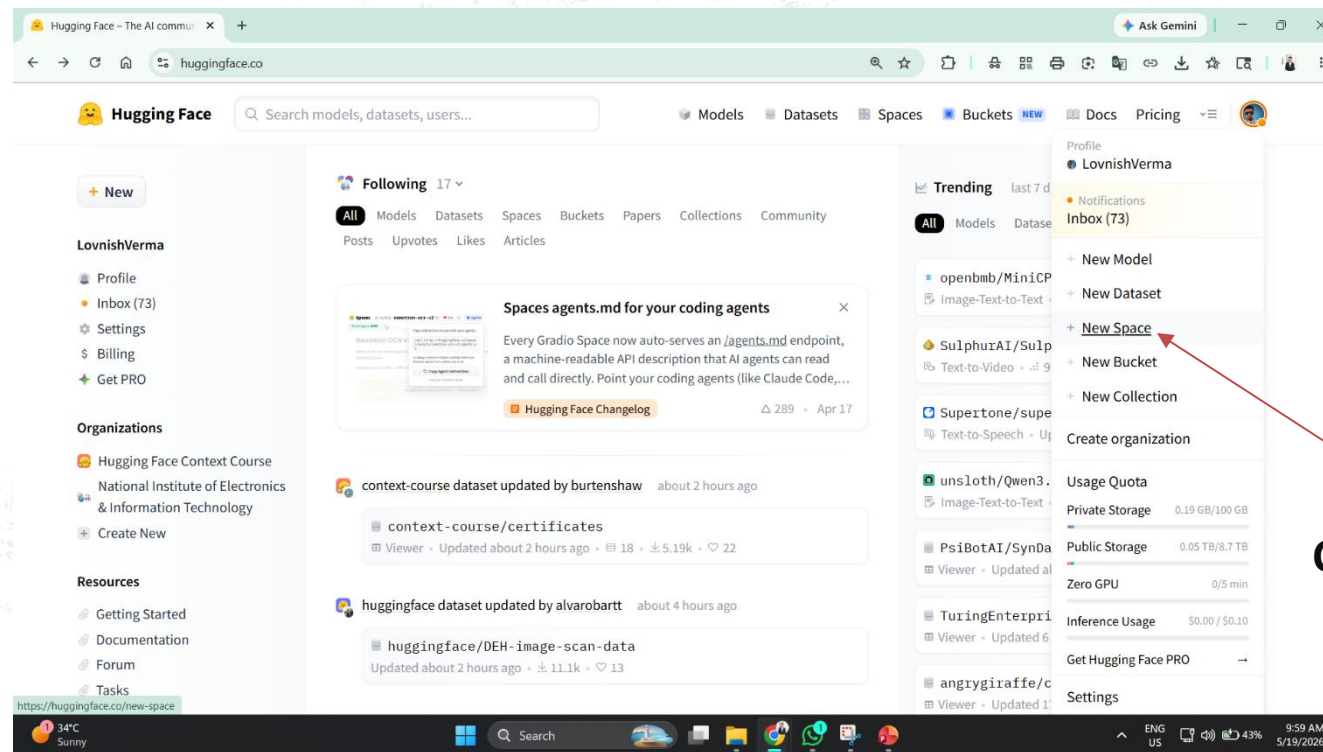


"Users love MongoDB because it offers the fastest time to value compared to any other DBMS technology".

Create a TODO App using MongoDB and Python Flask Framework On <https://huggingface.co/>

- **Set Up Huggingface Space:**

- Sign up or Login on huggingface.co
- Go to huggingface.co homepage after logging in and click on **Create a New Space**.



Create a New Space.

Create a TODO App using MongoDB and Python Flask Framework On Huggingface.co

- Enter Space Name
- Select Docker as SDK / Keep Template Blank



Create a new Space

Spaces are Git repositories that host application code for Machine Learning demos.
You can build Spaces with Python libraries like [Gradio](#), or using [Docker](#) images.

Owner: LovnishVerma / Space name: **New Space name**

Short description: Short Description

License: License

Select the Space SDK
You can choose between [Gradio](#), [Docker](#), or [Static](#) to host your Space.

Gradio
4 templates

Docker
17 templates

Static
6 templates

Choose a Docker template:

Blank

Streamlit

JupyterLab

Argilla

Space Dev Mode BETA PRO subscribers

Dev mode allows you to remotely access your Space using SSH or VS Code. [Learn more.](#)

☐ Enable dev mode

✦ [Subscribe to PRO](#) to use dev mode.

☒ **Public**

Anyone on the internet can see this Space. Only you can commit.

☐ **Protected** PRO subscribers

The app is publicly accessible via its URL, but the code and repo are private. Only you can see and commit to the code.

✦ [Subscribe to PRO](#) to use protected Spaces.

☐ **Private**

Only you can see and commit to this Space.

① **Tip:** Install the official [HF CLI Skill](#) so your AI agents can manage Spaces directly.

Create Space

Create a TODO App using MongoDB and Python Flask Framework On Huggingface

Scroll down a little and Copy all this code

Create your Dockerfile:

*# Read the doc: <https://huggingface.co/docs/hub/spaces-sdks-docker>
you will also find guides on how best to write your Dockerfile*

```
FROM python:3.9

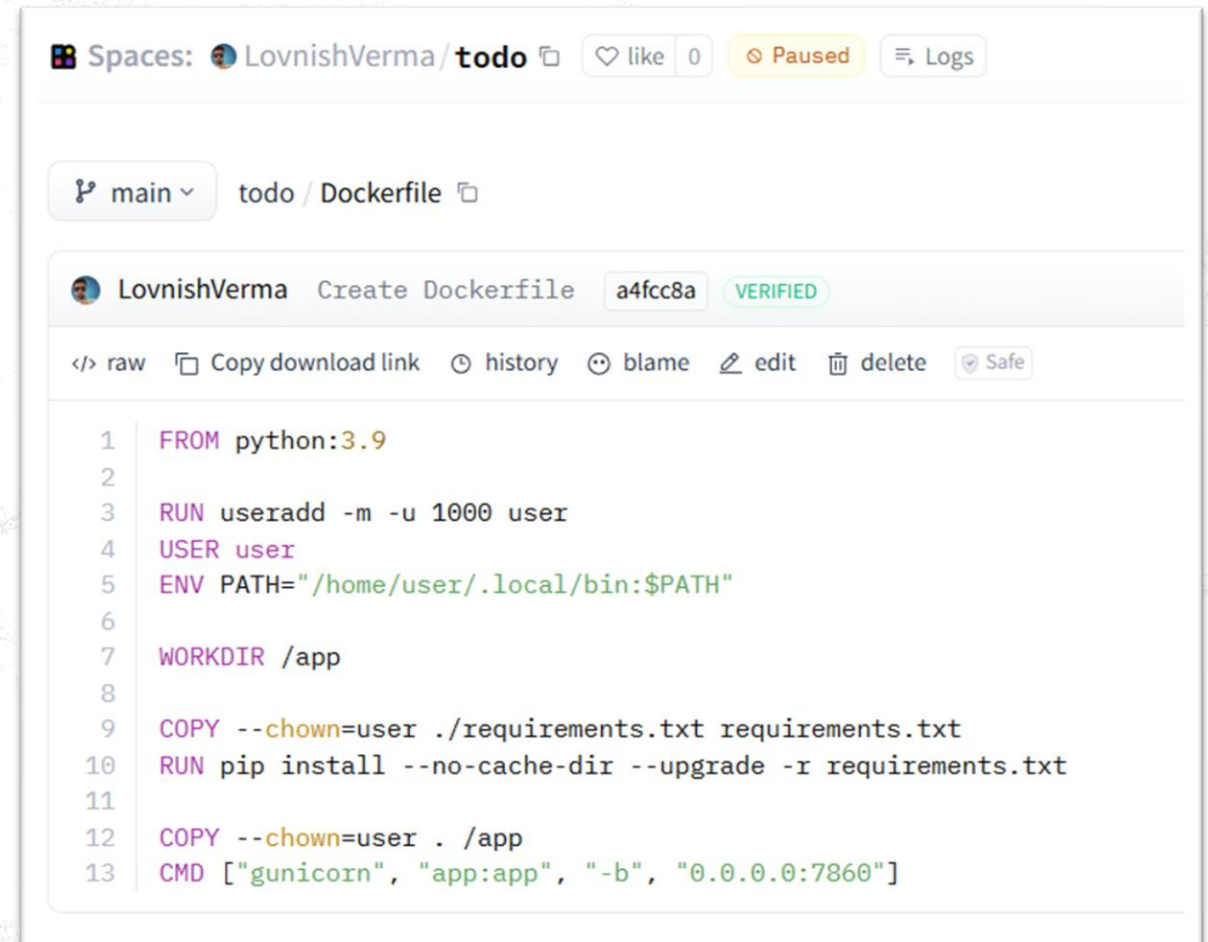
RUN useradd -m -u 1000 user
USER user
ENV PATH="/home/user/.local/bin:$PATH"

WORKDIR /app

COPY --chown=user ./requirements.txt requirements.txt
RUN pip install --no-cache-dir --upgrade -r requirements.txt

COPY --chown=user . /app
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
```

Hint Alternatively, you can **create the Dockerfile** file directly in your browser.



The screenshot shows the Huggingface Spaces interface for a space named 'todo' by user 'LovnishVerma'. The space is currently 'Paused'. The selected branch is 'main' and the file being viewed is 'Dockerfile'. The Dockerfile content is displayed with line numbers 1 through 13. The code includes instructions for setting up a Python 3.9 environment, creating a user, setting the working directory to /app, copying requirements, installing dependencies, and running the application with uvicorn on port 7860.

```
1 FROM python:3.9
2
3 RUN useradd -m -u 1000 user
4 USER user
5 ENV PATH="/home/user/.local/bin:$PATH"
6
7 WORKDIR /app
8
9 COPY --chown=user ./requirements.txt requirements.txt
10 RUN pip install --no-cache-dir --upgrade -r requirements.txt
11
12 COPY --chown=user . /app
13 CMD ["gunicorn", "app:app", "-b", "0.0.0.0:7860"]
```

Click on create the Dockerfile

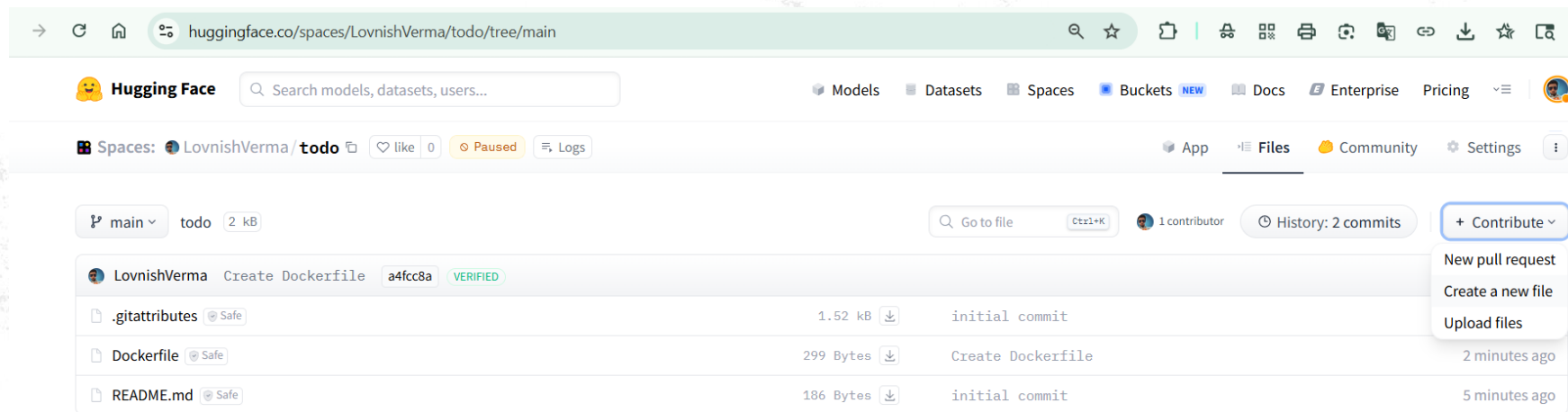
CMD ["gunicorn", "app:app", "-b", "0.0.0.0:7860"]



Create a TODO App using MongoDB and Python Flask Framework On Huggingface



- Click on **Files** and then **Contribute** and then **Create a new file**



PROJECT STRUCTURE

/my-todo-app

- app.py
- requirements.txt
- Dockerfile
- /templates
 - index.html



Create a ToDO App using MongoDB and Python Flask Framework on Huggingface



app.py This file connects to MongoDB, fetches tasks, and allows you to add or delete them.

todo/ app.py

```
Edit Preview
1 from flask import Flask, render_template, request, redirect, url_for
2 from pymongo import MongoClient
3 from bson.objectid import ObjectId
4 import os
5
6 app = Flask(__name__)
7
8 # Connect to MongoDB using a secure environment variable
9 # We will set MONGO_URI in Hugging Face settings later
10 MONGO_URI = os.getenv("MONGO_URI", "mongodb+srv://test:test@cluster0.sxc11.mongodb.net/?retryWrites=true&w=majority")
11 client = MongoClient(MONGO_URI)
12
13 # Select the database and collection
14 db = client.todo_database
15 todos_collection = db.todos
16
17 @app.route('/')
18 def index():
19     # Fetch all tasks from MongoDB
20     tasks = list(todos_collection.find())
21     return render_template('index.html', tasks=tasks)
22
23 @app.route('/add', methods=['POST'])
24 def add():
25     task_content = request.form.get('task-content')
```

Get it for
free from
mongodb
atlas

Create a ToDO App using MongoDB and Python Flask Framework On Huggingface

Edit index.html file add following code:

todo/ templates/index.html

```
Edit Preview
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>MongoDB To-Do App</title>
7   <style>
8     body { font-family: Arial, sans-serif; margin: 40px auto; max-width: 400px; text-align: center; }
9     input[type="text"] { padding: 10px; width: 65%; border: 1px solid #ccc; border-radius: 4px; }
10    button { padding: 10px 15px; cursor: pointer; background: #198754; color: white; border: none; border-radius: 4px; }
11    button:hover { background: #157347; }
12    ul { list-style-type: none; padding: 0; }
13    li { background: #f8f9fa; margin: 10px 0; padding: 12px; display: flex; justify-content: space-between; border-radius
14    a { color: red; text-decoration: none; font-weight: bold; }
15  </style>
16 </head>
17 <body>
18   <h2>🐉 MongoDB To-Do List</h2>
19
20   <form action="/add" method="POST">
21     <input type="text" name="task" placeholder="Enter a new task..." required>
22     <button type="submit">Add Task</button>
23   </form>
```

Create a TODO App using MongoDB and Python Flask Framework On Huggingface

- Add list of all required libraries in **requirements.txt** file.

todo/ requirements.txt

Edit	Preview
1	Flask==3.0.0
2	pymongo==4.6.1
3	gunicorn

Commit new file

Create requirements.txt

Edit

Preview

Add an extended description...

Upload images, audio, and videos by dragging in the text input, pasting, or [click](#)

Commit new file to main

Cancel

Steps to get your MongoDB Atlas URL

- **1. Sign Up / Log In to MongoDB Atlas**
- Visit MongoDB Atlas.
- If you don't already have an account, sign up for a free account.
- Log in to your account.
- **2. Create a New Cluster**
- Once logged in, click "**Build a Cluster**" or "**Create**".
- Select a **M0** free tier cluster
 - Set Cluster Name
 - Choose a cloud provider (e.g., AWS, Azure, GCP).
 - Select a region (choose one closer to your location for better performance).
 - Click "**Create Deployment**".

Steps to get your MongoDB Atlas URL

- **Create a Database User**
- Create a username and password (e.g., admin / pass123police).

Don't add @ in your password it will cause error later on.

- Then Click on **Create Database User**.
- Then click on Choose connection method.

i You'll need your database user's credentials in the next step. Copy the database user password.

Username

admin

Password

pass123police

HIDE

 Copy

Create Database User

Close

Choose a connection method

Steps to get your MongoDB Atlas URL

- **Create a Database User**
- Create a username and password (e.g., admin / pass123police).
- Then **click on Choose connection method.**

i You'll need your database user's credentials in the next step. Copy the database user password.

Username

admin

Password

pass123police

HIDE

 Copy

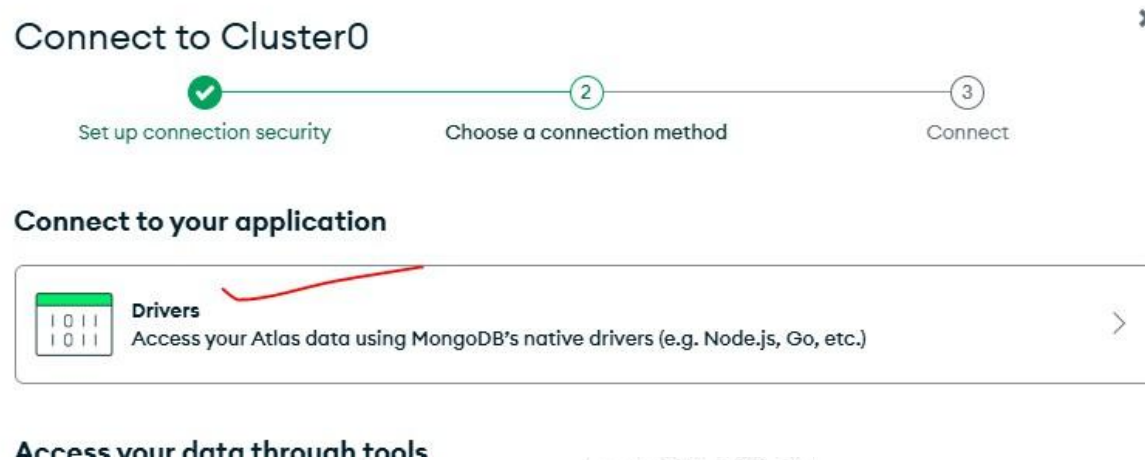
Create Database User

Close

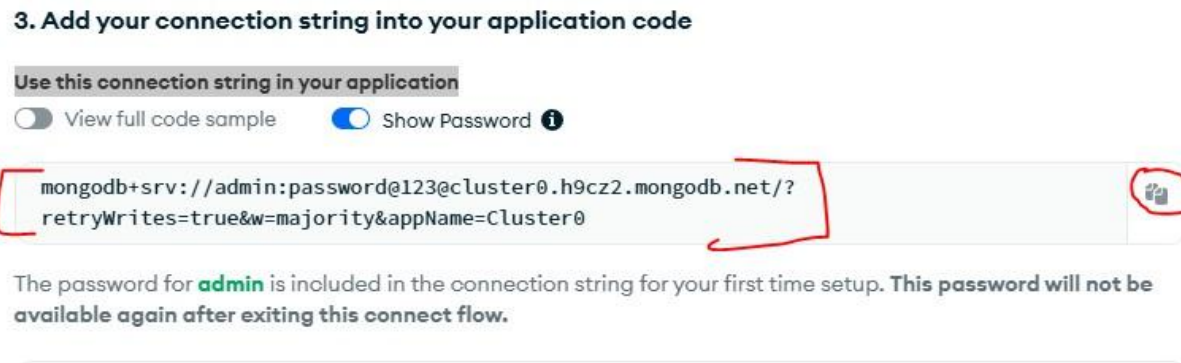
Choose a connection method

Steps to get your MongoDB Atlas URL

- Click on Drivers:

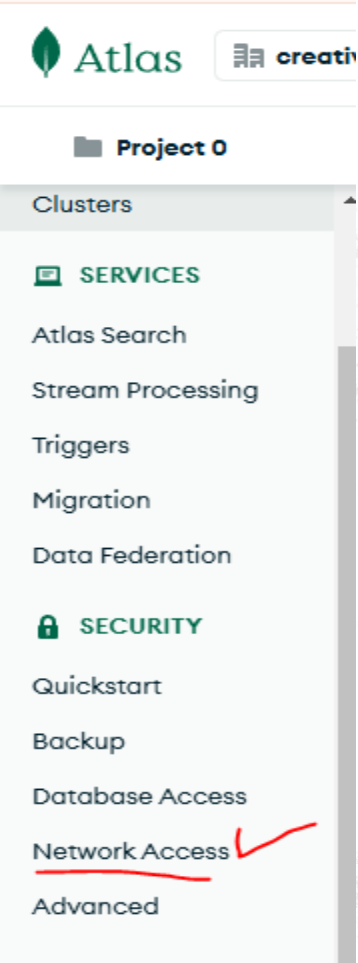


- You'll See your connection string copy it and save it somewhere safe:



Steps to get your MongoDB Atlas URL

- **Allow Network Access**
- Go to "Network Access" in the left sidebar.
- Click "Add IP Address".
 - Choose "Allow Access from Anywhere" (recommended for development only). This adds 0.0.0.0/0 to allow access from any IP address.
 - Alternatively, add specific IPs if you know your development machine's public IP.
- Save changes by clicking Confirm.



+ADD IP ADDRESS

ALLOW ACCESS FROM ANYWHERE

Access List Entry:

Comment:

☐ This entry is temporary and will be deleted in



Create a TODO App using MongoDB and Python Flask Framework On Huggingface



- Add your Connection URL in your flask app edit app.py file

```
app = Flask(__name__)
app.secret_key = "nielit_secret_key_here" # Required for flash messages

# MongoDB connection
MONGO_URI = os.getenv("MONGO_URI", "mongodb+srv://test:test@cluster0.sxcil.mongodb.net/?retryWrites=true&w=majority")
client = MongoClient(MONGO_URI)

# Database and Collection setup
db = client.todo_app # Database name
todos = db.todos     # Collection name
```

While creating a file write **templates/index.html** it will create templates folder and inside it index.html file

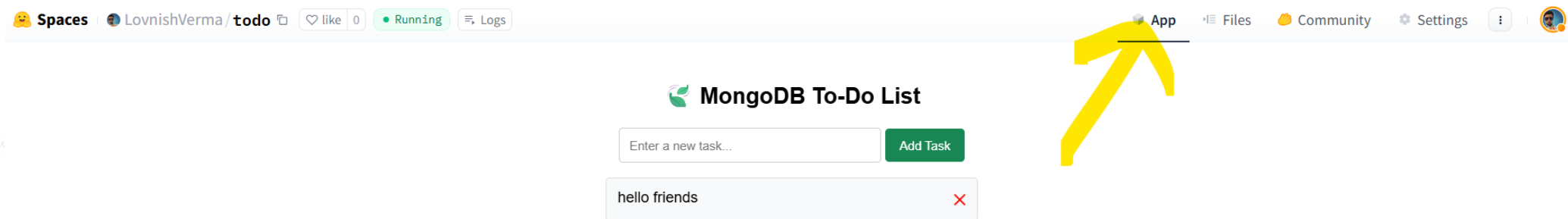




Create a ToDO App using MongoDB and Python Flask Framework On Huggingface



- Let's View our app Click on app



- You can also see your app by visiting <https://yourusername-spacename.hf.space>



Create a ToDO App using MongoDB and Python Flask Framework On Huggingface



Let's Add Some Tasks to test our newly created app.

- Enter Task and Click on Add Task.
- Wow as we can see your app is working Fine.

MongoDB To-Do List

hello friends



See Collection on MongoDB Atlas

To See your Collection on MongoDB Atlas

- Click on Clusters.
- Then Click on Your Cluster Name.
- Then Click on Collections.

CREATIVE_GRAPHICS_BAZAAR'S ORG - 2025-01-07 > PROJECT 0 > DATABASES

Cluster0

VERSION 8.0.4 REGION AWS Mumbai (ap-south-1) CLUSTER TIER M0 Sandbox (General)

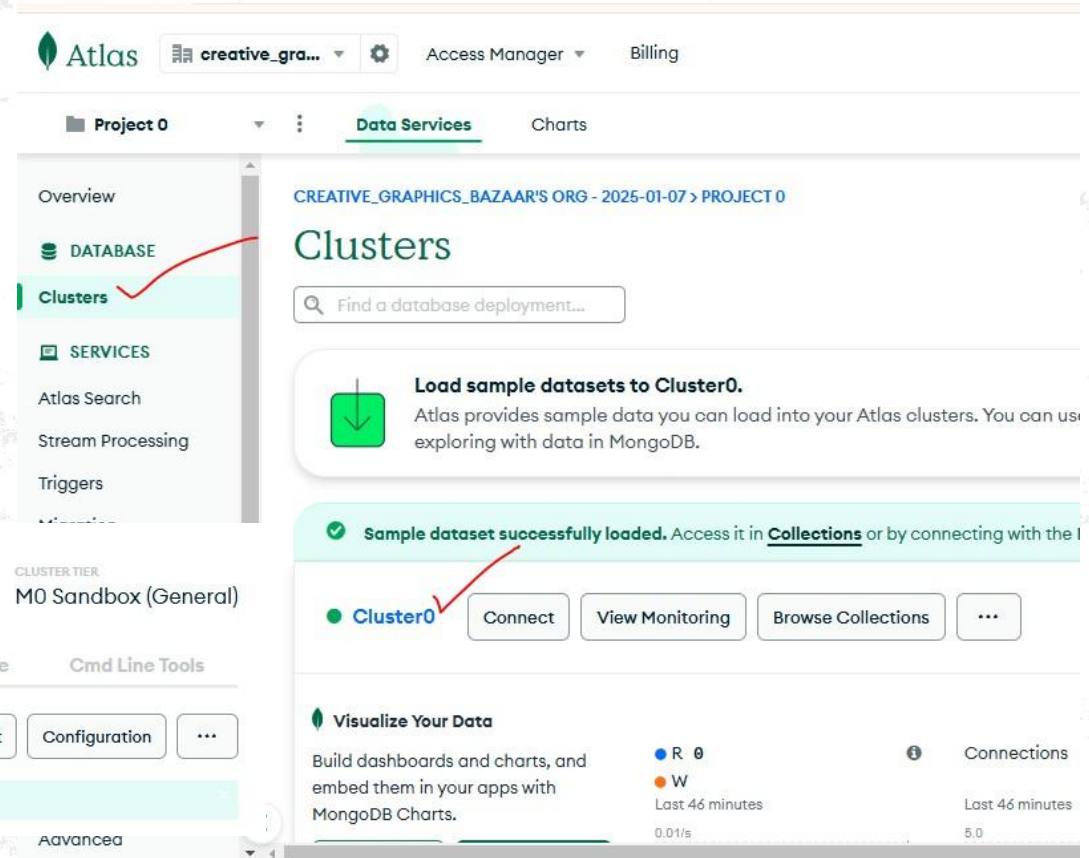
Overview Real Time Metrics Collections Atlas Search Performance Advisor Online Archive Cmd Line Tools

SANDBOX NODES REPLICASET

Connect Configuration ...

Sample dataset successfully loaded. Access it in Collections or by connecting with the MongoDB Shell.

Advanced

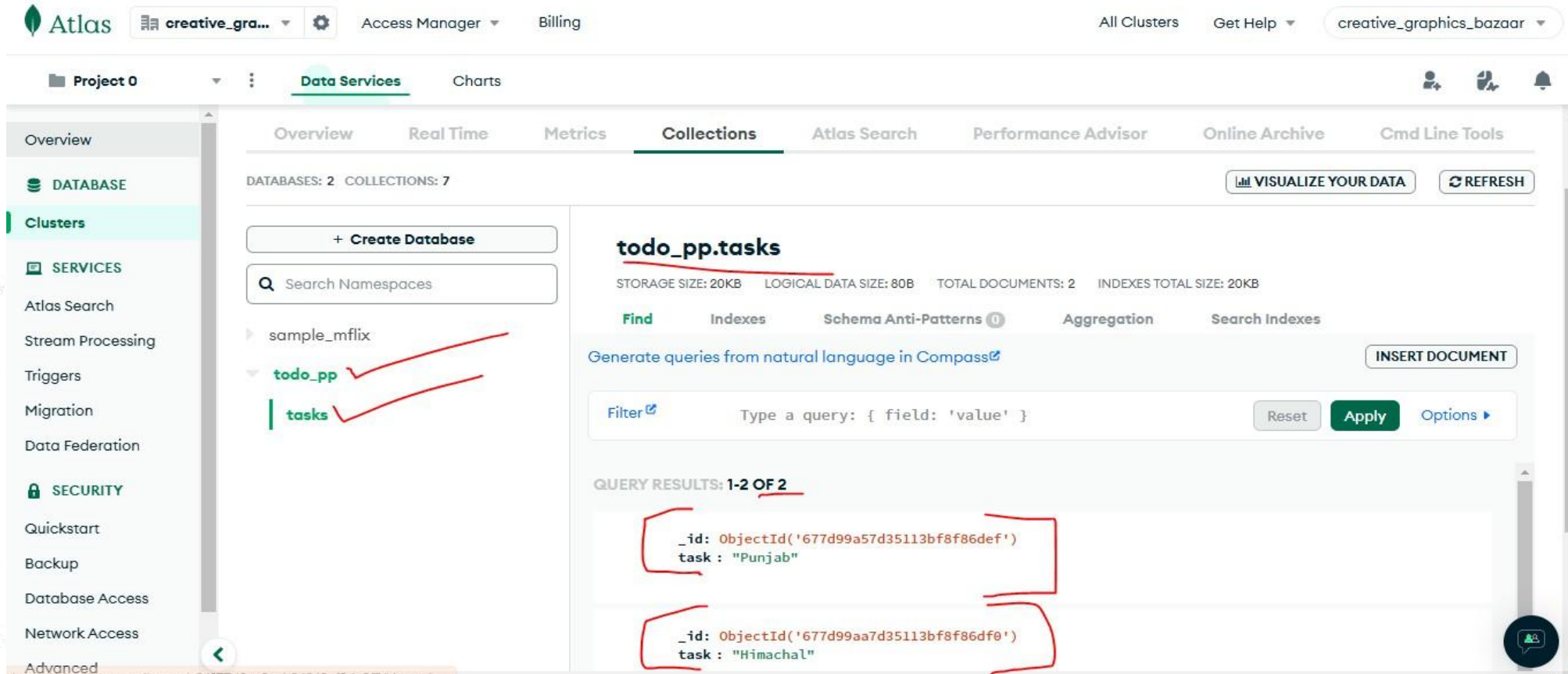


The screenshot shows the MongoDB Atlas interface. At the top, there's a navigation bar with 'Atlas', a dropdown menu showing 'creative_gra...', and links for 'Access Manager' and 'Billing'. Below this, a breadcrumb trail shows 'Project 0' > 'Data Services' > 'Charts'. The main content area is titled 'Clusters' and includes a search bar 'Find a database deployment...'. A green box with a download icon and text 'Load sample datasets to Cluster0. Atlas provides sample data you can load into your Atlas clusters. You can use exploring with data in MongoDB.' is visible. Below this, a green banner states 'Sample dataset successfully loaded. Access it in Collections or by connecting with the MongoDB Shell.' The 'Cluster0' section shows a green status icon, the name 'Cluster0', and buttons for 'Connect', 'View Monitoring', 'Browse Collections', and a three-dot menu. At the bottom, there's a 'Visualize Your Data' section with a description and a table showing read and write operations.

Visualize Your Data	Read (R)	Write (W)	Connections
Build dashboards and charts, and embed them in your apps with MongoDB Charts.	0	0	0
	Last 46 minutes	Last 46 minutes	Last 46 minutes
	0.01/s	0.01/s	5.0

See Collection on MongoDB Atlas

Click on your DB name and then your Collection Name to view all your Collections.



The screenshot shows the MongoDB Atlas web interface. The left sidebar contains navigation options: Overview, DATABASE, Clusters, SERVICES, Atlas Search, Stream Processing, Triggers, Migration, Data Federation, SECURITY, Quickstart, Backup, Database Access, Network Access, and Advanced. The main panel is titled 'Project 0' and shows 'Data Services' and 'Charts'. The 'Collections' tab is selected, displaying 'DATABASES: 2' and 'COLLECTIONS: 7'. A search bar for namespaces is present. The 'todo_pp' database is expanded, showing the 'tasks' collection. The 'tasks' collection details are shown, including 'STORAGE SIZE: 20KB', 'LOGICAL DATA SIZE: 80B', 'TOTAL DOCUMENTS: 2', and 'INDEXES TOTAL SIZE: 20KB'. The 'Find' tab is active, showing a query filter bar with the text 'Type a query: { field: 'value' }'. Below the filter, the 'QUERY RESULTS: 1-2 OF 2' are displayed, showing two documents: one with 'task: "Punjab"' and another with 'task: "Himachal"'. Red brackets and checkmarks are used to highlight the database and collection names in the sidebar and the query results.

Enjoy your own Personal ToDo App

What's Next:

- Add CSS Styling to improve App appearance.
- Add BootStarp to make it Mobile Fiendly.
- Add Advance Features like aggregation and make your app like <https://github.com/lovnishverma/ToDo-List-Flask-PyMongo> this improved version of app with better Styling (UI) and MongoDB aggregation.
- Keep Learning Try Creating Similar apps using MongoDB and Flask on Huggingface.

MongoDB To-Do List

hlo guys

×

tea

×

sugar

×

Live Demo



<https://huggingface.co/spaces/LovnishVerma/todo>